

Kraken

RAPPORT DE SESSION

19 mai 2026 • Durée : 1h52 • Coach : Théo

Participants : Pierre, Emeric, Manuel, Charlotte, Mika

🦑 TEMPS FORTS DE LA SESSION

Session technique menée à 90 % par un partage d'écran de Pierre, qui déroule son système complet d'intégration produit. Pas de hotseat niche ni de masterclass formelle cette semaine : l'essentiel est dans la mécanique d'automatisation et les échanges outils.

Les moments clés à ne pas manquer dans le replay :

- ▶ **00:00** Pierre – Passage d'Airtable à NocoDB : un tableur dynamique auto-hébergé sur VPS
- ▶ **00:09** Débat hébergement – Ressources, VPS Hostinger, Raspberry Pi et logique self-host
- ▶ **00:13** Le match N8N / Make / Claude – Quel socle d'automatisation pour l'intégration
- ▶ **00:22** Scraping – NPM dédié AliExpress vs scraper universel GitHub (bypass antibot)
- ▶ **00:25** NPM vs NPX (Manuel) – Paquets, exécution et mises à jour expliqués simplement
- ▶ **00:42** Sécurité – Cookies de session, hack Discord et Proton Pass / alias e-mail
- ▶ **00:53** Démo live – Le pipeline d'intégration produit de Pierre, étape par étape
- ▶ **01:40** Conseil du coach – Lancer maintenant, figner ensuite

✂ DÉCRYPTAGE — LE SYSTÈME DE PIERRE

► NocoDB, le tableur qui remplace Airtable

Pierre a migré d'Airtable vers **NocoDB**, un tableur dynamique open source qu'il auto-héberge sur son VPS (version gratuite, sans limite de colonnes). Pour lui, une ligne = un produit. Trois avantages concrets par rapport à Google Sheets : plusieurs images dans une même cellule, des feuilles reliées entre projets (SAV, sourcing...), et un chargement nettement plus rapide qu'Airtable qui laguait quand il poussait le volume.

Côté ressources, c'est léger : environ 1 % de CPU et 2 Go de RAM mobilisés. De quoi tourner sans souci, même sur un petit serveur. Le coach a relancé l'idée de l'héberger sur un Raspberry Pi à domicile, par goût du self-host (« l'appareil que je gère moi-même, que je peux customiser »), mais a conclu que pour Pierre, déjà équipé d'un VPS, ça n'a pas d'intérêt.

Le bénéfice de fond : tes données t'appartiennent, tout est chez toi, et l'outil est open source.

► Le match N8N / Make / Claude

Échange central de la session, déclenché par Manuel qui débute sur Make. La position de Pierre et du coach : **N8N est plus complet que Make** sur le long terme, même si son interface déroute au départ. Deux arguments pratiques reviennent.

Outil	Verdict	Ce qui ressort
N8N	Recommandé	Auto-hébergé sur VPS, gratuit. Accepte des nœuds de code (JavaScript, Python).
Make	Limitant	Crédits API, et pas de code custom : récupérer la donnée des premiers modules devient un enfer dès qu'il y a des branches.
Claude / Claude Code	Complémentaire	Génère et corrige le code via MCP. Très fort pour le scraping ; s'adapte quand un truc casse.

Le nœud JavaScript de N8N permet de récupérer directement les variables des premiers modules, là où Make bloque Pierre depuis longtemps. Emeric apporte la nuance inverse : son système sous Claude (skills + orchestrateurs) a été plus rapide à monter, mais demande des réajustements manuels par fiche. Le partage converge sur un point d'équilibre que Pierre résume bien.

« À la fin, ça fait les mêmes choses, que tu passes par Claude ou par N8N. Si tu lui expliques les mêmes choses, il fait les mêmes choses. »

— Pierre

► Le scraping : NPM dédié vs scraper universel

Emeric utilise un package NPM spécifique à AliExpress, appelé depuis un script Python via subprocess : le module scrape la page, renvoie du JSON, et Python le traite. Sa limite est structurelle — si AliExpress change son HTML, le package casse tant qu'il n'est pas mis à jour.

Pierre, lui, teste un **scraper universel trouvé sur GitHub** (54 étoiles, gratuit, avec Claude parmi les contributeurs). Il fonctionne sur n'importe quel site, contourne les protections antibot type Cloudflare, et affiche des benchmarks très rapides. Lui ne vise que AliExpress et 1688. Son installation passe par Python (pip), donc Claude peut le lancer et le piloter directement.

Pourquoi ce mélange Python / JavaScript ? Manuel l'explique clairement : une page web n'est que HTML, CSS et JavaScript. Le scraping consiste à interroger le DOM (Document Object Model), ce que JavaScript fait nativement et pas Python. Le choix de Python en orchestrateur est historique chez Emeric (cohérence avec ses autres skills et les MCP, eux aussi en Python).

► Le pipeline d'intégration produit (démon live)

Le cœur du partage : un système construit sur environ un an, découpé en workflows déclenchés par des webhooks (un bouton = un workflow) pour garder le contrôle à chaque étape.

1. Scraping & fusion. Le workflow scrape AliExpress et 1688, combine les deux (l'un complète l'autre), passe les images en OCR via Replicate (moins d'un centime l'image) pour récupérer le texte incrusté, puis nettoie la donnée avec Gemini pour ne garder que l'essentiel. Résultat : un JSON complet, ce que Pierre appelle sa **donnée first party**.

2. Remplissage du tableau. Gemini, avec un output JSON structuré, répartit automatiquement les données colonne par colonne : poids, tailles, litrage, nombre de poches, waterproof, catégories principale et secondaire.

3. Images. Téléchargées (préférence 1688, qualité supérieure), rangées dans Google Drive avec une nomenclature par produit. Un workflow dédié enlève le fond (Replicate), ajoute une ombre (Nano Banana 2), upscale, redimensionne en 1200x1200, compresse et range dans le dossier final.

4. Nom produit & mots-clés. Le titre est généré à partir de la donnée first party (rien n'est inventé) en piochant dans une base statique de mots-clés long-tail issus de Semrush — marques exclues, intention transactionnelle uniquement, triés une fois pour toutes lors de la recherche de niche.

5. Intégration finale. Le plus gros workflow analyse en temps réel les ID de catégories de la boutique, détecte la catégorie adaptée, génère la fiche SEO (8000 caractères, une cinquantaine de métafields) plus une description Merchant Center reformulée, distingue produit simple ou variable, et envoie le tout vers WooCommerce (ou Shopify, en changeant juste les modules d'entrée/sortie). Un calcul de marge par produit, alimenté par le prix d'achat scrappé, est intégré directement — sur le modèle du tableau partagé par Stéphanie.

Le coach a salué un travail « clean et entier », en insistant sur la valeur des détails accumulés. Pierre, lui, recentre sur ce qui compte vraiment.

« Le plus important, c'est vraiment la donnée que tu prends en entrée. Si tu prends le contexte que tu as déjà scrappé, tu auras une qualité de rédaction qui n'a rien à voir. »

— Pierre

⚙️ PRINCIPES TRANSVERSAUX & RAPPELS

Au-delà de la démon, plusieurs enseignements valables pour tout le monde sont ressortis des échanges.

NPM et NPX, démystifiés (Manuel) — NPM (Node Package Manager) est l'environnement qui exécute le JavaScript à partir des modules déclarés dans le fichier package.json ; c'est lui qu'on utilise en phase de développement et de débogage. NPX exécute un paquet déjà installé sous forme de binaire — plutôt pour la mise en production, plus rapide. Côté Python, l'équivalent est pip install / pip update.

Garder ses packages à jour — bonne pratique : commencer par un npm update avant d'installer. Astuce concrète : mettre *latest* dans le package.json plutôt qu'un numéro de version figé, pour rester toujours à jour. Un point à vérifier dans ses skills Claude — sachant qu'Emeric a constaté que sa skill de scraping était en Python,

donc concernée par pip et non par NPM.

Sécurité des comptes — la digression sur le hack Discord de Pierre (via un logiciel cracké) a rappelé le mécanisme des cookies de session : ils évitent de se reconnecter en permanence, mais quiconque les récupère peut usurper la session, parfois même en contournant la double authentification.

« Si je récupère les cookies de ta page, je me fais passer pour toi sur le serveur et je récupère tes informations. »

— Manuel

D'où les recommandations : ne jamais réutiliser le même mot de passe, utiliser un générateur, et passer par un gestionnaire comme **Proton Pass** (gratuit, ou environ 2-3 €/mois pour les alias e-mail dynamiques qui isolent ton vrai mail du spam et des fuites).

Lancer plutôt que figner — le fil rouge de la fin de session. Le coach a poussé Pierre (et Mika) à ne pas perdre de temps à migrer un système qui fonctionne déjà (Airtable vers NocoDB, N8N vers Claude). Ces refontes sont à garder pour le prochain site.

« Ton objectif, ça sera vraiment de lancer et pas de figner. Et après, tu fignoleras. »

— Théo

Emeric a complété par une mise en garde pour les débutants : le système de Pierre est un aboutissement, pas un point de départ.

« Si quelqu'un doit commencer, faut surtout pas se lancer là-dedans. Sinon, dans 6 mois, tu es encore en train d'essayer de comprendre le truc. »

— Emeric

🚧 PLAN D'ACTION PAR MEMBRE

🚧 Pierre

PRIORITAIRE

- ⚡ Lancer le site maintenant : objectif = mise en ligne, pas perfection. Le SAV et les ajustements viendront ensuite.
- ⚡ Tester le scraper universel GitHub et faire un compte-rendu au groupe (fiabilité, bypass antibot, AliExpress + 1688).
- ⚡ Finaliser l'intégration produit à 100 % (les images restantes) sur le site actuel avant tout autre chantier.

SECONDAIRE

- Garder N8N pour l'image et la rédaction (bien rodé) ; réserver le passage 100 % Claude au scraping, où c'est un vrai gain.
- Réserver la refonte dynamique (fichier statique unique par niche) au prochain site, pas à celui-ci.
- Montrer une page produit complète finie au groupe quand le site sera en ligne.

🚧 Manuel

PRIORITAIRE

- ⚡ Tester la stack N8N + Claude + Claude Code + VPS + NocoDB plutôt que de rester bloqué sur Make.

- ⚡ **Demander à Claude comment il gère les mises à jour NPM (automatique, ou à déclencher) pour clarifier le fonctionnement.**

SECONDAIRE

- Revoir la vidéo module 4 (intégration produit) et l'adapter de Shopify à WooCommerce.
- Remonter au coach tout point qui ne marche pas côté WooCommerce, pour qu'il fasse une partie dédiée.
- Ajouter un npm update en début d'instruction pour garantir des packages à jour.

🚢 Emeric

PRIORITAIRE

- ⚡ **Ajouter à son système l'étape de téléchargement et de stockage ordonné des images, en s'inspirant du pipeline de Pierre.**
- ⚡ **S'appuyer sur l'OCR / lecture d'images pour décider lesquelles retoucher, plutôt que des appels payants à vide.**

SECONDAIRE

- Reprendre les bonnes idées du système de Pierre sans tout refonçonner ; ses skills et orchestrateurs restent valables.
- Continuer à recalculer les volumes des sacs par les dimensions plutôt que de faire confiance aux infos AliExpress.

★ Tous les membres

PRIORITAIRE

- ⚡ **Prioriser le lancement sur le figolage : un site en ligne qui tourne vaut mieux qu'un système parfait jamais sorti.**

SECONDAIRE

- Pour scraper une page, filer directement le scraper GitHub à Claude : il lira la doc et se débrouillera.
- Sécuriser ses comptes : gestionnaire de mots de passe + alias e-mail (Proton Pass), jamais le même mot de passe partout.
- Récupérer les liens des deux scrapers partagés dans le chat / sur Skool.

◆ CHIFFRES CLÉS ÉVOQUÉS

Indicateur	Valeur	Contexte
VPS Hostinger KVM2	env. 10 €/mois	2 cœurs CPU, 8 Go RAM — socle de Pierre
NocoDB — ressources	1 % CPU / 2 Go RAM	Très léger, tournerait sur Raspberry Pi
Raspberry Pi 5 — RAM 8 Go	env. 200 €	Prix tiré par la RAM (1 Go : env. 50 €)
Scraper universel GitHub	54 étoiles	Gratuit, bypass antibot, multi-sites
OCR image (Replicate)	< 0,01 €	Par image traitée
Proton Pass	env. 2-3 €/mois	Version payante avec alias e-mail
Fiche produit SEO	8000 caractères	Générée par workflow
Métafields par fiche	env. 50	UX title, short title, avantages...
Images bloc / variations	10 / 4	Valeurs par défaut du tableau
Resize images	1200 x 1200	Après upscale et compression
Catalogue actuel Pierre	env. 100 produits	Site en cours de finalisation
RapidAPI — quota	100 requêtes/mois	Limite de l'ancienne méthode 1688
Marge cible	x3 environ	Prix d'achat scrappé automatiquement
Temps de build du système	env. 1 an	Aboutissement, pas point de départ

► VISIONNEZ LE REPLAY COMPLET

Ce rapport est un condensé. Les échanges complets, les nuances et les réponses aux questions sont dans le replay.